

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: T012	Name: Suyash Karnale
Class: TYIT	Batch: 1
Date of Assignment: 14-07-2026	Date/Time of Submission: 14-07-2026

Part A - Theory :

C# Delegate

What is a Delegate?

A **Delegate** is a special type in C# that stores the reference (address) of a method. It allows a method to be called indirectly through a delegate object. A delegate can call any method that has the **same return type and parameters** as the delegate declaration.

Why are Delegates Used?

- To call methods indirectly.
- To increase code flexibility.
- To implement callbacks.
- Used in event handling and GUI programming.

Real-Life Example

Think of a **TV Remote**.

- TV = Method
- Remote = Delegate

Instead of pressing buttons on the TV, we use the remote. Similarly, instead of calling a method directly, we use a delegate.

Syntax of Delegate

```
delegate returnType DelegateName(parameter_list);
```

Example

```
delegate void Operation(int a, int b);
```

Steps to Use a Delegate

Step 1: Declare the delegate

```
delegate void Operation(int a, int b);
```

Step 2: Create methods having the same signature

```
public void Add(int a, int b)
{
    Console.WriteLine(a + b);
}
```

Step 3: Create a delegate object

```
Operation obj;
```

Step 4: Assign a method to the delegate

```
obj = Add;
```

Step 5: Invoke the delegate

```
obj(10, 20);
```

or

```
obj.Invoke(10, 20);
```

Part B - Program

- a) Create methods add(), multiply(), subtract(), divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

1. Define a delegate with two integer parameters.
2. Create a **Calculator** class containing methods: Add, Sub, Multiply, Division.
3. In **Main()**, create an object of **Calculator**.
4. Declare a delegate reference.
5. Assign delegate to each method one by one.
6. Invoke the delegate with values to execute the method.
7. Display the result of each operation.

Code:

```
using System;
```

```
delegate void Operation(int a, int b);

class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition=" + (a + b));
    }
    public void Sub(int a, int b)
    {
        Console.WriteLine("Subtraction=" + (a - b));
    }
    public void Multiply(int a, int b)
    {
        Console.WriteLine("Multiply=" + (a * b));
    }
    public void Division(int a, int b)
    {
        Console.WriteLine("Division=" + (a / b));
    }
}

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        Operation obj;

        obj = c.Add;
        obj(10,5);

        obj = c.Sub;
        obj(10,5);

        obj = c.Multiply;
        obj(10,5);

        obj = c.Division;
        obj(10,5);

        Console.WriteLine("Suyash Karnale T012");
    }
}
```

Output:

```
Addition=15
Subtraction=5
Multiply=50
Division=2
Suyash Karnale T012
```

b) Write a program using multicast delegate.

Algorithm:

Algorithm:

- 1) Define delegate → `delegate void Message();`
- 2) Create class `Demo` → methods: `Welcome()`, `Learn()`, `ThankYou()`.
- 3) In `Main()` → create object `Demo d`.
- 4) Declare delegate reference → `Message obj;`
- 5) Assign methods to delegate → `obj = d>Welcome; obj += d.Learn; obj += d.ThankYou;`
- 6) Invoke delegate → `obj();` executes all three methods in sequence.
- 7) Print extra message → "`Suyash`" is displayed after delegate calls.

Code:

```
using System;
```

```
delegate void Message();
```

```
class Demo
```

```
{
```

```
    public void Welcome()
```

```
    {
```

```
        Console.WriteLine("Welcome Students");
```

```
    }
```

```
    public void Learn()
```

```
    {
```

```
        Console.WriteLine("Learn Delegates");
```

```
    }
```

```
    public void ThankYou()
```

```
    {
```

```
        Console.WriteLine("Thank You");
```

```
    }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
Demo d = new Demo();

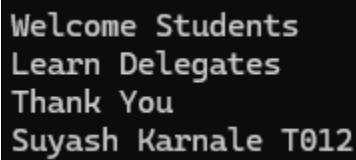
Message obj;

obj = d.Welcome;
obj += d.Learn;
obj += d.ThankYou;

obj();

Console.WriteLine("Suyash Karnale T012");
}
```

Output:



```
Welcome Students
Learn Delegates
Thank You
Suyash Karnale T012
```

- c) Create a class **BankAccount** with **AccountNumber** and **Balance**. Implement property for **Balance** with validation (**Balance** cannot be negative).

Algorithm:

1. Define class **BankAccount**
 - Property: **AccountNumber** (public).
 - Field: **balance** (private).
 - Property: **Balance** with getter and setter.
 - Setter allows only non-negative values.
2. In **Main()**
 - Create object **BankAccount b**.
 - Set **AccountNumber = 101**.
 - Set **Balance = 5000**.
 - Print account number and balance.
 - Try setting **Balance = -1000** → shows validation message.
 - Print **"Suyash"**.

Code:

```
using System;
```

```
class BankAccount
{
    public int AccountNumber { get; set; }

    private double balance;

    public double Balance
    {
        get
        {
            return balance;
        }
        set
        {
            if (value >= 0)
                balance = value;
            else
                Console.WriteLine("Balance cannot be negative.");
        }
    }
}

class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();

        b.AccountNumber = 101;
        b.Balance = 5000;

        Console.WriteLine("Account Number : " + b.AccountNumber);
        Console.WriteLine("Balance : " + b.Balance);

        b.Balance = -1000;
        Console.WriteLine("Suyash Karnale T012");
    }
}
```

Output:

```
Account Number : 101
Balance : 5000
Balance cannot be negative.
Suyash Karnale T012
```

Curiosity Question

1. A college wants to calculate student performance.

- a. Create methods to
- b. Calculate Total Marks
- c. Calculate Percentage
- d. Display Grade
- e. Display Pass/Fail

Note: Call each method using a delegate.

Algorithm:-

Code:-

```
using System;

class Student
{
    public int m1, m2, m3, total;
    public double percentage;

    public delegate void StudentDelegate();

    public void CalculateTotal()
    {
        total = m1 + m2 + m3;
        Console.WriteLine("Total Marks = " + total);
    }

    public void CalculatePercentage()
    {
        percentage = total / 3.0;
        Console.WriteLine("Percentage = " + percentage + "%");
    }

    public void DisplayGrade()
    {
        if (percentage >= 75)
            Console.WriteLine("Grade = A");
        else if (percentage >= 60)
            Console.WriteLine("Grade = B");
        else if (percentage >= 50)
            Console.WriteLine("Grade = C");
        else
            Console.WriteLine("Grade = D");
    }

    public void DisplayResult()
    {
        if (percentage >= 40)
            Console.WriteLine("Result = Pass");
    }
}
```

```
        else
            Console.WriteLine("Result = Fail");
    }
}

class Program
{
    static void Main()
    {
        Student s = new Student();

        // Input marks
        Console.Write("Enter marks for 3 subjects: ");
        s.m1 = int.Parse(Console.ReadLine());
        s.m2 = int.Parse(Console.ReadLine());
        s.m3 = int.Parse(Console.ReadLine());

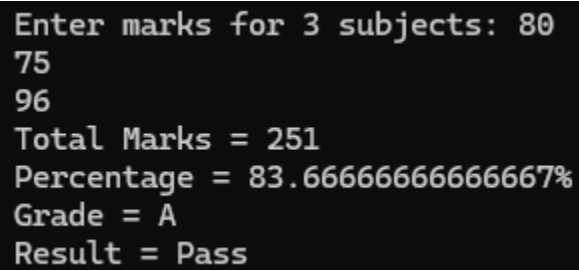
        // Create delegate instance
        Student.StudentDelegate sd;

        // Add methods to delegate (multicast)
        sd = s.CalculateTotal;
        sd += s.CalculatePercentage;
        sd += s.DisplayGrade;
        sd += s.DisplayResult;

        // Invoke delegate
        sd();

        Console.ReadLine();
    }
}
```

Output:-

A screenshot of a terminal window showing the output of the program. The text is as follows:

```
Enter marks for 3 subjects: 80
75
96
Total Marks = 251
Percentage = 83.66666666666667%
Grade = A
Result = Pass
```

2. A college wants to automate the admission process. When the user clicks **Confirm Admission**, several operations must happen automatically. Design a delegate-based solution.

Students decide:

- a. Verify Documents

- b. Generate Roll Number
- c. Allocate Division
- d. Send SMS

Note: For above operation just print a statement.

Algorithm:-

Code:-

```
using System;

delegate void AdmissionDelegate();

class Admission
{
    public void VerifyDocuments()
    {
        Console.WriteLine("Documents Verified");
    }

    public void GenerateRollNumber()
    {
        Console.WriteLine("Roll Number Generated");
    }

    public void AllocateDivision()
    {
        Console.WriteLine("Division Allocated");
    }

    public void SendSMS()
    {
        Console.WriteLine("SMS Sent to Student");
    }
}

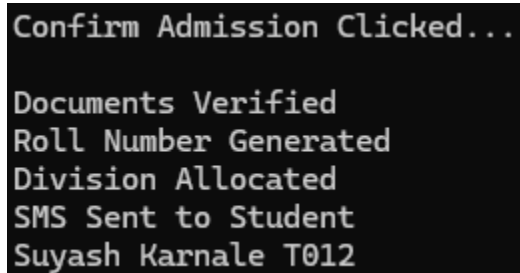
class Program
{
    static void Main()
    {
        Admission a = new Admission();

        AdmissionDelegate obj;

        obj = a.VerifyDocuments;
        obj += a.GenerateRollNumber;
        obj += a.AllocateDivision;
```

```
obj += a.SendSMS;  
  
Console.WriteLine("Confirm Admission Clicked...\n");  
  
obj(); // Calling all operations using delegate  
  
Console.WriteLine("Suyash Karnale T012");  
}  
}
```

Output:-

A screenshot of a console window with a black background and white text. The output consists of five lines: "Confirm Admission Clicked...", "Documents Verified", "Roll Number Generated", "Division Allocated", and "SMS Sent to Student". The last line, "Suyash Karnale T012", is not visible in the screenshot.

```
Confirm Admission Clicked...  
  
Documents Verified  
Roll Number Generated  
Division Allocated  
SMS Sent to Student
```